DIPARTIMENTO DI ELETTRONICA,
INFORMAZIONE E BIOINGEGNERIA

Politecnico di Milano

Machine Learning (Code: 097683)
June 20, 2023

Surname:
Name:
Student ID:
Row: Column:

Time: 2 hours Maximum Marks: 33

- The following exam is composed of **8 exercises** (one per page). The first page needs to be filled with your **name, surname and student ID**. The following pages should be used **only in the large squares** present on each page. Any solution provided either outside these spaces or **without a motivation** will not be considered for the final mark.
- During this exam you are **not allowed to use electronic devices**, such as laptops, smartphones, tablets and/or similar. As well, you are not allowed to bring with you any kind of note, book, written scheme, and/or similar. You are also not allowed to communicate with other students during the exam.
- The first reported violation of the above-mentioned rules will be annotated on the exam and will be considered for the final mark decision. The second reported violation of the above-mentioned rules will imply the immediate expulsion of the student from the exam room and the **annulment of the exam**.
- You are allowed to write the exam either with a pen (black or blue) or a pencil. It is your responsibility to provide a readable solution. We will not be held accountable for accidental partial or total cancellation of the exam.
- The exam can be written either in **English** or **Italian**.
- You are allowed to withdraw from the exam at any time without any penalty. You are allowed to leave the room not early than half the time of the duration of the exam. You are not allowed to keep the text of the exam with you while leaving the room.
- **Three of the points will be given on the basis on how quick you are in solving the exam.** If you finish earlier than 30 min before the end of the exam you will get 3 points, if you finish earlier than 20 min you will get 2 points and if you finish earlier than 10 min you will get 1 point (the points cannot be accumulated).
- The box on Page 10 can only be used to complete the Exercises 7, and/or 8.

Ex. 1	Ex. 2	Ex. 3	Ex. 4	Ex. 5	Ex. 6	Ex. 7	Ex. 8	Time	Tot.
/ 7	/ 7	/ 2	/ 2	/ 2	/ 2	/ 4	/ 4	/ 3	/ 33

Exercise 1 (7 marks)

Describe the Bayesian Linear Regression approach and compare it to the least squares method.

Exercise 2 (7 marks)

Illustrate the PAC bound for consistent learners (train error equal to zero) and its applications.

Exercise 3 (2 marks)

Consider the following snippet of code:

```
1 x = zscore(dataset['petal-length'].values).reshape(-1, 1)
2 y = zscore(dataset['class'].values)
3
4 n_samples = len(x)
5 Phi = np.ones((n_samples, 2))
6 Phi[:, 1] = x.flatten() # the second column is the feature
7 w = inv(Phi.T @ Phi) @ (Phi.T.dot(y))
```

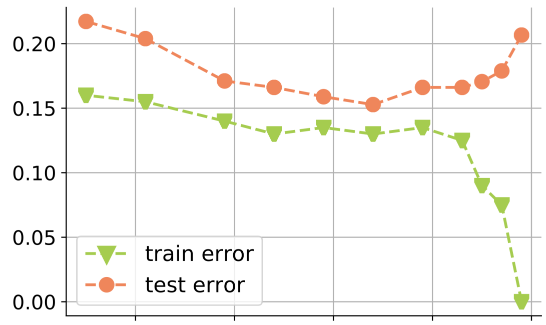
1. Describe the operations executed and the purpose (which problem and which method has been used) of the snippet reported above.
2. Do you think the methods used above are appropriate for the considered problem? If not, propose an alternative approach. If yes, propose other approaches to solve the same problem.
3. Are there any issue intrinsic to Line 7? Describe the issue and propose a method to deal with such a problem.

1. The snippet is applying OLS to a classification problem.
2. Logistic Regression or other classification methods will be more suited than OLS.
3. In line 7, the inversion of $\Phi^T\Phi$ is feasible only if the matrix is not singular. This problem can be avoided using ridge regression.

Exercise 4 (2 marks)

The figure below was produced by changing a hyperparameter (on the x-axis) in a classifier/regressor on a given training/test dataset.

For each one of the following methods along with the description of how hyperparameters was changed (from left to right), tell if the picture could have been produced by it:



1. Ridge regression, increasing regularization parameter λ.
2. k-nearest neighbor classifier, decreasing number of neighbors k.
3. Logistic regressor, increasing the number of principal components extracted from the dataset.
4. SVM, decreasing the number of features selected by a filtering method.

1. NO: Increasing the regularization would increase the training error, therefore the graph cannot be corresponding to this case.
2. YES: The more the K parameter is small the more it is prone to overfit the original dataset. With $K = 1$ all the data in the training set are perfectly classified.
3. YES: The more the number of features we are considering, the more the model il complex and prone to overfitting.
4. NO: It would decrease the complexity of the model and therefore the training error should increase.

Exercise 5 (2 marks)

Tell if the following expressions are valid kernels, motivating your answer:

1. $k(x, x') = ak_1(x, x') + bk_2(x', x)$, where $k_1(\cdot, \cdot)$ and $k_2((\cdot, \cdot)$ are valid kernels
2. $k(x, x') = ||x||_2^2 + ||x||_2||x'||_2$
3. $k(x, x') = \frac{|xx'|}{xx'} + 2$ for $x, x' \in [-1\ 1]$
4. $k(x, x') = x^\top Ax'$ with $A = \begin{bmatrix} 3 & 1 \\ 1 & -4 \end{bmatrix}$ and $x, x' \in \mathbb{R}^2$

The formation of the kernel should follow the Mercer's theorem conditions (continuous, symmetric and semidefinite positive function).

1. As long as $a, b > 0$ we are assured that the formation of the above kernel provides a valid kernel.
2. The kernel form is not symmetric (due to the first term).
3. The kernel is not continuous in $x = 0$ for each $x' \in [-1\ 1]$
4. Since the matrix A is not semidefinite positive ($det(A) = 3 \cdot (-4) - 1 \cdot 1 = -13 < 0$) it cannot identify a valid kernel.

Exercise 6 (2 marks)

You are an ML specialist working in an industrial company that builds automatic manipulator systems in charge of collecting objects in a 2D environment. Consider the following ML problems:

1. Estimate the time needed by the manipulator to collect a specific object, based on the object location, and having access to a dataset of the past executions of the manipulator.

2. Learn the manipulator control policy that minimizes the amount of time needed to collect all the objects in the 2D environment, having the possibility to interact with the real system.

Model the two problems described above as ML problems, specifying the data (features and target or states, actions, and reward), the problem class, and a specific method to solve them.

1. SUPERVISED LEARNING - REGRESSION. The features are the position (for instance, x, y) of the object to be collected and the target is the time needed for the collection. A method to solve it is KNN.

2. REINFORCEMENT LEARNING - CONTROL. The state is given by the positions of the objects in the 2D environment and the position of the robotic manipulator, the actions are the forces to be applied to the joints of the manipulator (or directly the position to be reached), and the reward is the opposite of the time needed to collect one object (or a negative constant for every time step). The problem is episodic, since it terminates as all objects have been collected. A possible method to solve it is Q-Learning (provided that we discretize state and action spaces).

Exercise 7 (4 marks)

Assume you need to learn the optimal parameter of a perceptron classifier, starting from a parameter vector $\mathbf{w}^{(0)} = (1\ 0)^\top$ and a learning rate set to $\alpha = 1$. You are given the following dataset:

$$\mathbf{x}_1 = (2\ 1)^\top \qquad t_1 = 1$$
$$\mathbf{x}_2 = (1\ -2)^\top \qquad t_2 = -1$$
$$\mathbf{x}_3 = (1\ 3)^\top \qquad t_3 = -1$$
$$\mathbf{x}_4 = (-3\ 2)^\top \qquad t_4 = 1$$

1. Compute the vector $\mathbf{w}$ once we run the gradient descent algorithm on the provided dataset once.
2. Do you think that the computed $\mathbf{w}$ is optimal for the given dataset?
3. Can you tell if the perceptron training procedure would converge eventually?
4. In general, given 4 distinct points with arbitrary labels in a 2D space, can they always be correctly classified by a perceptron?

1. The update prescribed by the online update given by the gradient descent algorithm is:

$$w^{(k+1)} = w^{(k)} - \alpha \nabla L(w) = w^{(k)} + \alpha x t_n$$

if the prediction of the perceptron is wrong $\hat{t}_n \neq t_n$. Therefore:

   (a) $\mathbf{x}_1$ correctly classified ($sign(\mathbf{x}_1^\top \mathbf{w}) = sign((2\ 1)(1\ 0)^\top) = sign(2) = 1 = t_1$), no update.

   (b) $\mathbf{x}_2$ misclassified ($sign(\mathbf{x}_2^\top \mathbf{w}) = sign((1\ -2)(1\ 0)^\top) = sign(1) = 1 \neq t_2$), therefore:

$$w^{(1)} = (1\ 0) + 1(1\ -2)(-1) = (0\ 2).$$

   (c) $\mathbf{x}_3$ misclassified, therefore:

$$w^{(2)} = (0\ 2) + 1(1\ 3)(-1) = (-1\ -1).$$

   (d) $\mathbf{x}_4$ correctly classified, no update.

2. Since the first sample is not yet correctly classified by the last vector $w^{(2)}$ the convergence has not yet achieved.
3. Since the four points are not linearly separable (you should draw them to prove it), the procedure will not converge.
4. NO, since the VC dimension of a linear classifier in 2D is 3, in general 4 points cannot be shattered by a perceptron in 2D.

Exercise 8 (4 marks)

Consider the following episode obtained by an agent interacting with an MDP having three states $\mathcal{S} = \{A, B, C\}$ and two actions $\mathcal{A} = \{l, r\}$,

$$(A, l, 1) \to (C, l, 2) \to (A, r, 0) \to (B, r, 5) \to (B, l, 0) \to (C, l, 0) \to (B, l, -2) \to (A, r).$$

Consider an initial state-action values $Q(s, a) = 0$ for every state-action pair, a learning rate $\alpha = 0.5$, and a discount factor $\gamma = 1$. Answer to the following questions providing adequate motivations.

1. Execute the *Q-learning* algorithm on the given episode (break ties using the first action listed in the action set).
2. Execute the *SARSA* algorithm on the given episode.
3. Provide the best policy according to the output of *SARSA*. Is it different from the one provided by *Q-learning*?

1. The formula fo Q-learning is $Q(s, a) \leftarrow Q(s, a) + \alpha(R + \gamma \max_{a'} Q(s', a') - Q(s, a))$, which in our case is:

$$\begin{aligned}
Q(A, l) &= 0 + 0.5(1 + 1 \cdot 0 - 0) = 1/2 \\
Q(C, l) &= 0 + 0.5(2 + 1 \cdot 0.5 - 0) = 5/4 \\
Q(A, r) &= 0 + 0.5(0 + 1 \cdot 0 - 0) = 0 \\
Q(B, r) &= 0 + 0.5(5 + 1 \cdot 0 - 0) = 5/2 \\
Q(B, l) &= 0 + 0.5(0 + 1 \cdot 5/4 - 0) = 5/8 \\
Q(C, l) &= 5/4 + 0.5(0 + 1 \cdot 5/2 - 5/4)) = 15/8 \\
Q(B, l) &= 5/8 + 0.5(-2 + 1 \cdot 1/2 - 5/8) = -7/16
\end{aligned}$$

2. The formula for SARSA is $Q(s, a) \leftarrow Q(s, a) + \alpha(R + \gamma Q(s', a') - Q(s, a))$, where a' is the currently chosen action. In our case:

$$\begin{aligned}
Q(A, l) &= 0 + 0.5(1 + 1 \cdot 0 - 0) = 1/2 \\
Q(C, l) &= 0 + 0.5(2 + 1 \cdot 0 - 0) = 1 \\
Q(A, r) &= 0 + 0.5(0 + 1 \cdot 0 - 0) = 0 \\
Q(B, r) &= 0 + 0.5(5 + 1 \cdot 0 - 0) = 5/2 \\
Q(B, l) &= 0 + 0.5(0 + 1 \cdot 1 - 0) = 1/2 \\
Q(C, l) &= 1 + 0.5(0 + 1 \cdot 1/2 - 1)) = 3/4 \\
Q(B, l) &= 1/2 + 0.5(-2 + 1 \cdot 0 - 1/2) = -3/4
\end{aligned}$$

3. The policy prescribed by SARSA is to do the following action in the corresponding states: $A \to l$, $B \to r$, and $C \to l$. Instead, Q-learning would suggest $A \to l$, $B \to r$, and $C \to l$. Therefore, their suggestions would be the same.